

A Comprehensive Comparative Analysis of CARLA and AWSIM: Open-Source Autonomous Driving Simulators

Zhihui Lai^{c,*}, Lee Le^c, Vihanga Silva^d, Thomas Bräunl^c

^a*The University of Western Australia, 35 Stirling Hwy, Crawley, 6009, WA, Australia*

^b*Western Power, 363 Wellington St, Perth, 6000, WA, Australia*

Abstract

The number of open-source autonomous driving simulators has been increasing in recent years, and many papers have surveyed the most popular options. However, there is a lack of in-depth and comprehensive comparison among the most advanced open-source simulators. This article provides a detailed quantitative and qualitative analysis of two of the most promising simulators for end-to-end testing: CARLA and AWSIM. The analysis focuses on various metrics, including the efficiency of simulation, the realism of physics simulation, scenario creation and testing, user-friendliness, and extendability. Based on our findings, we conclude that CARLA remains the best open-source simulator for end-to-end testing, while AWSIM shows advanced performance in specific areas, such as Lidar-based algorithms and Autoware development.

Keywords: Autonomous driving simulator, Unreal engine, Unity, CARLA, AWSIM

*Corresponding author.

Email addresses: `zhihui.lai@research.uwa.edu.au` (Zhihui Lai),
`23011365@student.uwa.edu.au` (Lee Le), `vihanga.silva@outlook.com` (Vihanga Silva), `thomas.braunl@uwa.edu.au` (Thomas Bräunl)

Preprint submitted to Simulation Modelling Practice and Theory

January 13, 2025

A Comprehensive Comparative Analysis of CARLA and AWSIM: Open-Source Autonomous Driving Simulators

Zhihui Lai^{c,*}, Lee Le^c, Vihanga Silva^d, Thomas Bräunl^c

^c*The University of Western Australia, 35 Stirling Hwy, Crawley, 6009, WA, Australia*

^d*Western Power, 363 Wellington St, Perth, 6000, WA, Australia*

1. Introduction

Autonomous driving simulations are crucial for supplementing and accelerating real-world testing, which can be difficult, dangerous, or even impossible in some cases [1, 2]. Over the past few years, the domain of simulators has witnessed a surge in the development of both commercial and open-source simulators. Interestingly, the number of open-source simulators has been on the rise [3]. The most popular commercial simulators currently include Waymo SimulationCity [4]; Cognata [5]; AVSimulation (SCANer) [6]; Simcenter Prescan [7]; CarSim [8]; IPG CarMaker [9]; MATLAB RoadRunner Scenario [10]; and Nvidia Drive Sim [11]. In contrast, the open-source simulators that have gained popularity are Gazebo [12], Webots [13], AirSim [14], CARLA [15], and LGSVL [16]. While these simulators help researchers study autonomous driving tasks, including perception, localization, planning, vehicle control, and 3D scene reconstruction, there is a lack of in-depth and comprehensive comparison among the most advanced open-source simulators.

This work fills the gap by performing a comprehensive comparative analysis of two simulators, CARLA and AWSIM [17], where the former is prominent in the domain of autonomous driving, and the latter is new but has a lot of potential. This analysis will involve performance testing (quantitative) and functionality comparison (quantitative and qualitative), offering insights

*Corresponding author.

Email addresses: zhihui.lai@research.uwa.edu.au (Zhihui Lai), 23011365@student.uwa.edu.au (Lee Le), vihangasilva@outlook.com (Vihanga Silva), thomas.braunl@uwa.edu.au (Thomas Bräunl)

for researchers and developers selecting simulators for autonomous driving applications.

When assessing the effectiveness of an autonomous driving simulator, it is essential to consider its efficiency, involving the degree of simplification and computational cost required to run the simulator and the time it takes to complete each simulation step. Simulators that need less computational power and memory are more accessible and cost-effective but may sacrifice realism and accuracy. Another critical consideration is the level of realism and fidelity it provides in replicating real-world driving scenarios, including factors such as the accuracy of the physics engine, the realistic behavior of other vehicles and pedestrians, and the authenticity of the sensor, environment, and weather conditions. A high-fidelity simulator can shrink the "Reality Gap" [18, 19], so the trade-off between simulator efficiency and fidelity is important [20]. Support for the scenario creation and testing is also crucial. Simulators with rich scenario libraries, construction tools, and well-designed traffic management systems are preferred. Besides, It is necessary to take user-friendliness into account. UI functionality, learning curve, community and support, and cost and licensing are legit indicators of how hard it is to learn a simulator. Finally, the extendability of a simulator is critical to reflect its bridging ability to AI algorithms, ROS [21], and other simulators. All of these criteria play a crucial role in determining the overall effectiveness of the simulation, which is essential for developing and testing autonomous vehicle systems.

The following sections are organized. Section 2 reviews the existing literature on autonomous driving simulators. Section 3 identifies and describes the criteria for the end-to-end testing of autonomous driving simulators and analyzes the comparison results. Section 4 introduces the conclusion and future works.

2. Related Works

Real-world self-driving experiments are expensive, time-consuming, and unsafe. It also requires a tremendous effort for regulatory approval with road agencies in most countries. Kalra et al. stressed that autonomous driving systems (ADSs) require hundreds of millions of miles of driving to reach the same level of safety as human drivers [22]. However, this is impossible with real-world testing alone. Therefore, simulators are pivotal for safely speeding up real-world testing and reducing hardware and time costs. Open-source sim-

ulators like Gazebo, Webots, CARLA, and LGSVL are the preferred choices in academia due to their active communities and low barriers to entry. The advantages and disadvantages of these simulators have been extensively discussed and analyzed in recent publications. Fadaie surveyed several simulators from an industrial perspective, including companies like Toyota, Waymo (Google), Uber, NVIDIA, Microsoft, HERE, Mapper.ai, BMW, Ford, and GM; he also summarized the simulator characteristics comprising data generation, scenario controllability, software extendability, component level interfacing, scenario parameterization, system observability, usability and sim management, software reusability, throughput (iteration speed), experiment replay, and behavior visualization [23]. Violate compared 18 simulators with filters including team support, documentation, fidelity, ease of customization, extendability (especially with Apollo [24] and Autoware [25]), and HD maps support [26]. Kaur et al. surveyed six simulators - CARLA, LGSVL, CarSim, Gazebo, Prescan, and Matlab/Simulink - to explore their functionalities and performance on perception, the multi-view geometry, path planning, vehicle control, map and scenario creation, 2D/3D Ground truth, team support, API flexibility, compatibility of different OS, scalability via a server multi-client architecture, and open-source [27].

Meanwhile, Zhou et al. conducted a similar study, but with the main difference of replacing Gazebo with AirSim and using simpler criteria focusing on HD map and scenario generation, support for different sensors, NPC controlling, and vehicle dynamic model [28]. Li et al. conducted a thorough survey of 46 simulators, classifying them into five primary categories: traffic flow, sensory data, driving policy, vehicle dynamics, and comprehensive simulators. This survey serves as a helpful guide for researchers to select the most appropriate simulator based on their specific requirements [3]. Although useful, these surveys only collected surface-level information and did not compare the simulators through experiments or analyze their documents in detail.

There is a need for an in-depth comparison between these simulators using the same experiments in the same environments. However, building the same environments and experiments in all the simulators is time-consuming. It is best to compare the most popular ones. Since the surveys from [26], [27], [28], and [3] all indicated that CARLA and LGSVL are the two most suitable simulators for end-to-end testing, it is ideal to do experiments within these two simulators. However, LGSVL has been suspended since 2022, so we found an alternative - AWSIM to replace LGSVL. AWSIM is an open-

source unity-based simulator designed for Autoware and extensible for other tasks. It doesn't have a mature ecology like CARLA; however, it has several advantages, such as a smooth learning curve, low computational cost, and time acceleration, that are worth attracting more researchers to join and develop this community.

This paper aims to conduct an in-depth, comprehensive comparative analysis on the same map in Unreal Engine 4 (UE4)-based CARLA and Unity-based AWSIM to compare their simulator performance for autonomous driving. Hence, it compares the game engines behind the two simulators—UE4 and Unity.

3. Methodology and Results

This section identifies a set of criteria that can serve as a metric to determine which simulators best suit the task at hand. The criteria for defining an excellent autonomous driving simulator can be divided into two general domains: quantitative and qualitative analysis. For quantitative criteria, we conduct experiments and collect results. For qualitative criteria and mixed criteria, we conduct discussions. All results are listed in table 3.

3.1. *Simulation platform setup*

Since 2020, The Renewable Energy Vehicle (REV) Project at The University of Western Australia (UWA) has purchased two second-hand EasyMile shuttle buses with drive-by-wire and sensor systems installed without any software. This project aims to build a complete software stack and improve current algorithms for autonomous driving in a closed environment and on public roads. We use two test locations, one at the UWA Crawley campus and one in the northern Perth suburb of Eglinton, the new Amberton Beach residential development, connecting the Stockland Sales Center to Amberton Beach and then returning. The total length of the route is approximately 4.1 kilometers, with a maximum allowable speed of 50 kilometers per hour on the road. We built digital versions of the UWA campus and Amberton Beach in RoadRunner and then exported them to CARLA and AWSIM to test autonomous driving algorithms. As an experimental scenario, the Amberton Beach map in this paper compares CARLA and AWSIM's performance. Fig. 1 is the Amberton beach map with a regular path. Hardware Setup: All experiments are conducted on an OMEN laptop with three Ubuntu systems

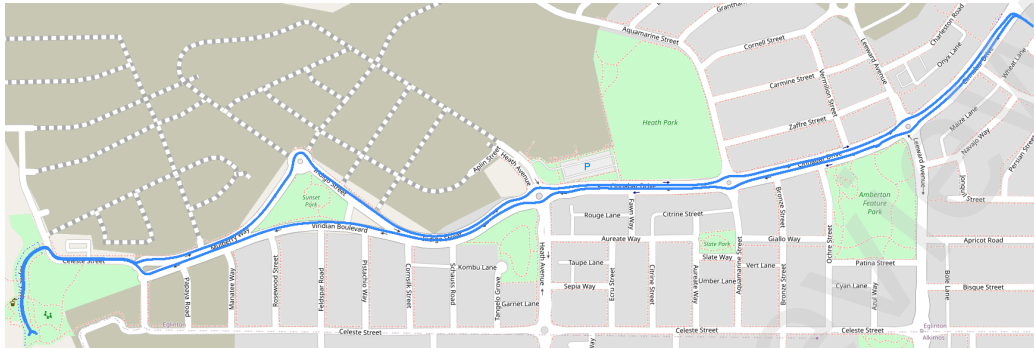


Figure 1: Amberton beach map with a regular path (scale 100m)

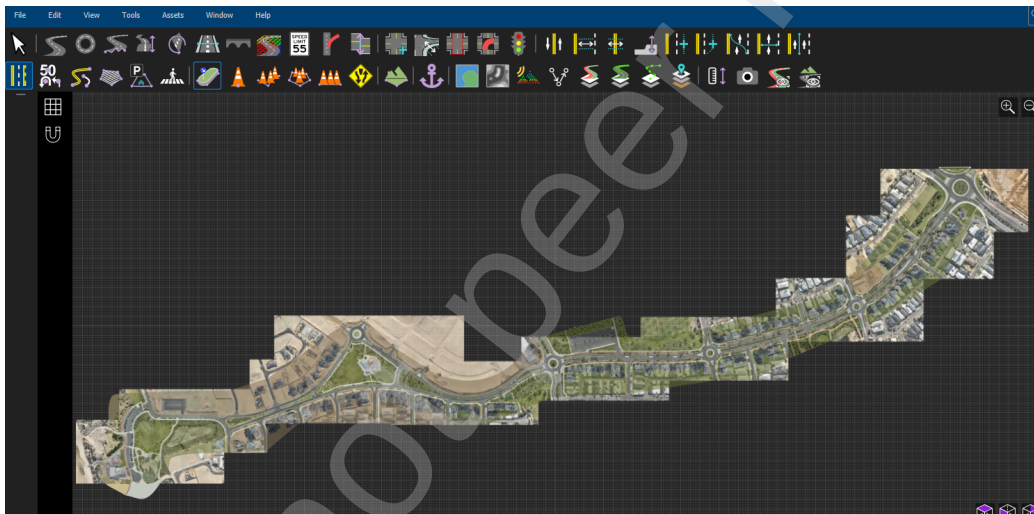


Figure 2: RoadRunner Eglinton map with aerial image

(18, 20, 22). The CPU is Intel i7-11800H, the RAM is 32 GB, and the GPU is NVIDIA GeForce RTX 3070.

RoadRunner Setup: Using the Eglinton aerial image, we built roads precisely, clustered plants roughly, and placed houses with similar template house models in RoadRunner R2024b. Fig. 2 is the RoadRunner Eglinton map with aerial image. Fig. 3 is the RoadRunner Eglinton map. Fig. 3 is the RoadRunner Eglinton map close shot.

CARLA Setup: We use the CARLA 0.9.12 standalone package on Ubuntu 20 and the CARLA 0.9.12 build from source on Ubuntu 18. Fig. 5 is the CARLA Eglinton map. Fig. 6 is the CARLA Eglinton close-up.



Figure 3: RoadRunner Eglinton map



Figure 4: RoadRunner Eglinton map close shot

AWSIM Setup: We use the AWSIM 1.3.1 in Unity editor 2021.1.7f1 and the AWSIM 1.3.1 executable on Ubuntu 22. Fig. 7 is the AWSIM Eglinton map. Fig. 8 is the AWSIM Eglinton close-up.

3.2. Efficiency of Simulation

To reflect the efficiency of the simulator, we use simplification/computational cost and processing speed.

3.2.1. Simplification/Computational Cost

Description. The simulator that requires less computational power for physics calculations has more spare capacity for running AI algorithms. To measure



Figure 5: CARLA Eglinton map



Figure 6: CARLA Eglinton close-up

the computational cost, we use GPU utilization, GPU used dedicated memory, CPU utilization, and CPU memory as metrics.

Experiments. Monitor and record GPU utilization, GPU used dedicated memory, CPU utilization, and CPU memory in 5 minutes at 20 Hz during identical simulation scenarios with its optimal delta time using Python performance monitoring modules: psutil and GPUutil. CARLA's optimal delta time is 0.05, while AWSIM's is 0.01666667, so changing the delta time may result in bad

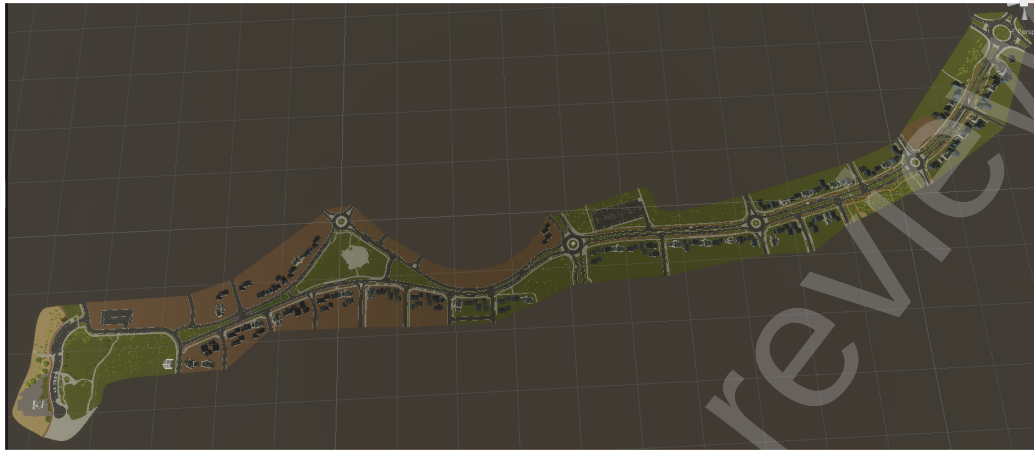


Figure 7: AWSIM Eglinton map

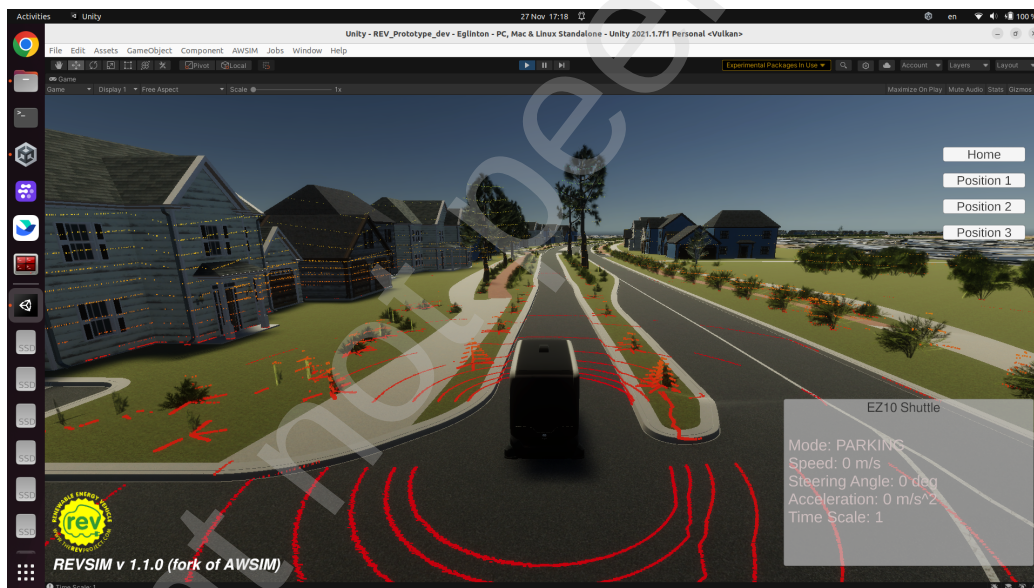


Figure 8: AWSIM Eglinton close-up

performance. Reducing CARLA delta time (fixed delta seconds) will lower the ROS publishing rates of camera and Lidar and increase frame drops. Increasing AWSIM delta time (fixed time step) will reduce the ROS publishing rates of camera and Lidar.

Results. Table 1 illustrates the Eglinton map simulator average performance testing results in 5 mins 20Hz (CARLA Delta Time = 0.05, AWSIM Delta Time = 0.01666667). AWSIM uses less GPU memory, CPU Util, and memory than CARLA, but it has the same or higher GPU Util. So AWSIM consumes less computing power than CARLA.

3.2.2. Processing Speed

Description. The simulator with a higher processing speed can accelerate the iteration of AI algorithms.

Experiments. Measure the real-time factor (the time taken for each simulation step in certain conditions).

Results. Table 1 and 2 show the Eglinton map simulator real-time factor. AWSIM can always keep up with real-time, but CARLA in asynchronous mode is too fast with frame drops and hard to control, and CARLA in synchronous mode is too slow with a low ROS topic publishing rate. Besides, AWSIM can change the time scale, making the real-time factor more flexible.

3.3. Realism of Physics Simulation

Evaluate how accurately each simulator models real-world physics, including vehicle dynamics, sensor simulation, and environmental interactions (e.g., weather and road conditions).

3.3.1. Vehicle Dynamics

Description. Accurately simulating vehicle dynamics is crucial for effective control algorithm development in autonomous vehicles. An ideal simulator must capture critical features such as tire-road interaction, multi-body dynamics, actuator modeling, and control systems to support advanced control strategies, vehicle design evaluation, and overall performance enhancement [29].

Discussion. Unity or Unreal game engines use NVIDIA PhysX pre-integrated, so CARLA, which is based on UE4, and AWSIM, which is based on Unity, have the same vehicle dynamics level. Trade-offs in NVIDIA PhysX prioritize visualization over precise physics behavior [29], so both have low vehicle dynamics performance. Still, CARLA supports integration with CarSim (not free), which has accurate vehicle dynamics, and hence, CARLA obtains a higher score than AWSIM.

Table 1: CPU, GPU and Real-time Factor Comparison Table (CPU: Intel i7-11800H; RAM: 32 GB; GPU: NVIDIA GeForce RTX 3070)

	Metrics	CARLA Async	CARLA Sync	AWSIM
All Sensors	GPU Utilization	39.55%	32.52%	37.85%
	GPU Used Dedicated Memory	30.30%	30.21%	16.98%
	Real-time Factor	128.79%	65.50%	100.00%
	CPU Utilization	25.13%	29.56%	30.02%
	CPU Memory	22.70%	22.34%	12.68%
Cam Only	GPU Utilization	40.16%	39.22%	66.36%
	GPU Used Dedicated Memory	30.27%	30.23%	9.48%
	Real-time Factor	133.17%	124.13%	100.00%
	CPU Utilization	16.54%	30.57%	31.05%
	CPU Memory	22.53%	22.31%	12.68%
Lidar Only	GPU Utilization	38.75%	30.19%	38.12%
	GPU Used Dedicated Memory	29.28%	29.23%	16.49%
	Real-time Factor	251.12%	116.02%	100.00%
	CPU Utilization	36.76%	32.47%	22.46%
	CPU Memory	22.54%	22.24%	11.07%
GPS and IMU Only	GPU Utilization	39.75%	38.79%	38.66%
	GPU Used Dedicated Memory	29.33%	29.24%	8.94%
	Real-time Factor	300.76%	265.69%	100.00%
	CPU Utilization	20.76%	33.51%	14.20%
	CPU Memory	22.38%	22.11%	10.40%

Table 2: One Lap Comparison Table (CPU: Intel i7-11800H; RAM: 32 GB; GPU: NVIDIA GeForce RTX 3070)

	CARLA Async	CARLA Sync	AWSIM
One Lap Real Finish Time (s)	848.8	1373	812.8
Real-time Factor	118.63%	59.61%	100.00%
Camera FPS	23.77	11.94	20
Lidar FPS	23.77	11.94	20

3.3.2. Sensor Simulation

Description. Effective sensor modeling is critical for autonomous vehicles, which rely on sensors for environmental perception. Simulators must accurately model sensors like Lidar, cameras, radar, IMU, and GNSS to account for sensor noise. Cameras and Lidar are the two most important and challenging sensors to simulate [30, 31], so we have more criteria for them.

Table 3: CARLA vs. AWSIM Comparison Table

Criteria	Weight	CARLA				AWSIM Unity	
		Raw	Score	Weighted	Raw	Score	Weighted
GPU Utilization	2	32.52%	1.000	2.000	37.85%	0.859	1.718
GPU Used Dedicated Memory	1	30.21%	0.562	0.562	16.98%	1.000	1.000
Real-time Factor	3	65.50%	0.655	1.965	100.00%	1.000	3.000
CPU Utilization	2	29.56%	1.000	2.000	30.02%	0.985	1.969
CPU Memory	1	22.34%	0.568	0.568	12.68%	1.000	1.000
Tire-Road Interaction	1	-	0.300	0.300	-	0.000	0.000
Multi-body Dynamics	1	-	0.300	0.300	-	0.000	0.000
Actuator and Powertrain Modelling	1	-	0.300	0.300	-	0.000	0.000
Integrated Control System Modeling	1	-	0.300	0.300	-	0.000	0.000
Vehicle Parametrization and Customization	1	-	0.300	0.300	-	0.000	0.000
GNSS	3	✓	1.000	3.000	✓	1.000	3.000
IMU	3	✓	1.000	3.000	✓	1.000	3.000
RGB Camera	10	✓	0.850	8.500	✓	0.950	9.500
Depth Camera	2	✓	1.000	2.000	-	0.000	0.000
Event/DVS Camera	2	✓	1.000	2.000	-	0.000	0.000
LiDAR	10	✓	0.500	5.000	✓	0.750	7.500
Radar	2	✓	1.000	2.000	✓	1.000	2.000
V2X	1	✓	1.000	1.000	✓	1.000	1.000
Night	4	✓	1.000	4.000	-	0.300	1.200
Cloudy	4	✓	1.000	4.000	-	0.300	1.200
Rainy	4	✓	0.800	3.200	-	0.200	0.800
Foggy	2	✓	0.800	1.600	-	0.200	0.400
Snowy	1	-	0.100	0.100	-	0.200	0.200
Lighting Conditions	5	✓	1.000	5.000	✓	0.800	4.000
Pedestrian Lights	4	✓	1.000	4.000	✓	1.000	4.000
Classic Trafficlights	5	✓	1.000	5.000	✓	1.000	5.000
Pedestrians	4	✓	1.000	4.000	✓	1.000	4.000
Cyclist/Motorcyclist	2	✓	1.000	2.000	-	0.000	0.000
Cars	5	✓	1.000	5.000	✓	1.000	5.000
Trucks	3	✓	1.000	3.000	✓	1.000	3.000
New Maps	10	✓	0.700	7.000	✓	0.800	8.000
New Vehicles	8	✓	0.800	6.400	✓	1.000	8.000
New Sensors With Templates	5	✓	0.800	4.000	✓	1.000	5.000
New Sensors Without Templates	3	✓	1.000	3.000	-	0.000	0.000
Number of Sample Maps	6	8	1.000	6.000	1	0.200	1.200
Number of Sample Vehicles	5	36	1.000	5.000	6	0.400	2.000
Number of Sample Pedestrians	3	11	1.000	3.000	1	0.200	0.600
Vehicle Motion	6	✓	0.850	5.100	✓	0.700	4.200
Pedestrian Motion	3	✓	1.000	3.000	✓	0.500	1.500
Traffic Lights Management	6	✓	1.000	6.000	✓	1.000	6.000
Modular Editing	2	✓	1.000	2.000	✓	1.000	2.000
Data Visualization	2	✓	1.000	2.000	✓	1.000	2.000
Installation	2	Slow	0.200	0.400	Fast	1.000	2.000
Basic Operations	2	Normal	0.500	1.000	Easy	1.000	2.000
Availability of documentation	3	Good	1.000	3.000	Good	0.600	1.800
Community Activity	3	High	1.000	3.000	Low	0.200	0.600
Licensing Fees	2	Open-source	1.000	2.000	Open-source	1.000	2.000
Hardware Requirements	2	Normal	0.800	1.600	High	0.500	1.000
ROS Integration	10	Foxy	0.500	5.000	Humble	0.800	8.000
Collision Detector	6	✓	1.000	6.000	-	0.000	0.000
Lane Invasion Detector	6	✓	1.000	6.000	-	0.000	0.000
Obstacle Detector	6	✓	1.000	6.000	-	0.000	0.000
Camera Semantic Segmentation	10	✓	1.000	10.000	-	0.300	3.000
Camera Instance Segmentation	8	✓	1.000	8.000	-	0.300	2.400
2D Bounding box	10	✓	1.000	10.000	-	0.300	3.000
3D Bounding box	8	✓	1.000	8.000	-	0.300	2.400
Optical flow	5	✓	1.000	5.000	-	0.000	0.000
Lidar Semantic Segmentation	10	✓	1.000	10.000	✓	1.000	10.000
Lidar Intensity	5	-	0.000	0.000	✓	1.000	5.000
Programming Languages	5	Python, C++	1.000	5.000	C#	0.500	2.500
Total	252	-	49.485	218.495	-	34.144	149.688

1. RGB camera: In simulations, particularly for autonomous driving, cameras can have post-effects that simulate real-world visual conditions. These effects can enhance realism and make sensor data closer to what a real-world camera might capture. The common post-effects

Table 4: CARLA vs. AWSIM Camera Post-effects Comparison Table

Post-Effect	CARLA	AWSIM
Bloom	✓	✓
Motion Blur	✓	✓
Depth of Field	✓	✓
Vignette	✓	✓
Lens Flare	✓	✓
Chromatic Aberration	✓	✓
Exposure Adjustment	✓	✓
HDR	✓	✓
Noise and Grain	✓	✓
Color Grading	✓	✓
Tone Mapping	✓	✓
Distortion	✓	✓
Glare and Halo	-	✓
Sharpening	-	✓
Anti-Aliasing	-	✓

Table 5: CARLA vs. AWSIM Lidar Fidelity Comparison Table

Lidar Fidelity Metrics	CARLA	AWSIM
Total Configurable Parameters	13	32
Configurable Point Cloud Density and range	✓	✓
Configurable Field of View (FoV)	✓	✓
Noise and Artifacts	✓	✓
Ghost Points (Multipath)	-	-
Material Reflection	-	✓
Weather Effects (Rain, Fog)	-	-
Sunlight Interference	-	-
Clipping Range	✓	✓
Occlusion Handling	✓	✓
Moving Object Detection	✓	✓
Deformation Effects	-	-
GPU support	-	✓
Point Cloud Format	XYZI	XYZIR/XYZIRCAEDT

applied are:

- (a) Bloom: Enhances bright areas in the image, mimicking the effect

of light scattering in the lens.

- (b) Motion Blur: Simulates blurring caused by the camera's motion or objects in the scene.
 - (c) Depth of Field: Blurs objects outside the camera's focal range, simulating the focus behavior of real cameras.
 - (d) Vignette: Darkens the corners of the image, simulating the natural light fall-off at the edges of a lens.
 - (e) Lens Flare: Creates streaks of light when the camera faces a bright light source, like the sun.
 - (f) Chromatic Aberration: Introduces slight color fringing at objects' edges, simulating lens imperfections.
 - (g) Exposure Adjustment: Adjusts the brightness dynamically based on the light conditions in the scene, mimicking auto-exposure in real cameras.
 - (h) HDR (High Dynamic Range): Enhances the contrast between dark and bright areas, improving the visibility of details in challenging lighting conditions.
 - (i) Noise and Grain: Adds random speckles or noise to simulate sensor noise in low-light conditions or older cameras.
 - (j) Color Grading: Alters the colors in the image to simulate different film stocks or artistic styles.
 - (k) Tone Mapping: Maps the high dynamic range of the scene into the limited range of a display, controlling how bright areas and shadows are rendered.
 - (l) Distortion: Simulates lens distortion, where straight lines appear curved depending on the lens type (e.g., fisheye or wide-angle).
 - (m) Glare and Halo: Creates glowing artifacts around bright light sources, replicating the effect of light scattering within the camera's optics.
 - (n) Sharpening: Enhances the image's clarity of edges and fine details.
 - (o) Anti-Aliasing: Smooths jagged edges in the image, which can occur due to the discrete nature of pixels.
2. Depth camera: Depth cameras are important for ADAS algorithm development because they provide extra depth information.
 3. Event/DVS Camera: Event cameras have advantages when capturing moving objects, making them crucial for ADAS algorithm development.
 4. Lidar: High-fidelity Lidar simulation should provide realistic point cloud data and reflect real-world challenges like noise, range limitations,

environmental conditions, and material reflections. The key metrics and methods for evaluating the fidelity of Lidar sensors are:

- (a) Configurable Point Cloud Density And Range: Configurable point cloud density (number of points per unit area) to match those of a real sensor.
- (b) Configurable Field of View (FoV): Verify that the simulated Lidar's horizontal and vertical fields of view and angular resolution match those of a real sensor.
- (c) Gaussian Noise: Analyze the noise pattern in the point cloud. Simulated Lidar should introduce some noise (similar to real Lidar), especially with longer ranges or reflective materials.
- (d) Ghost Points (Multipath): Check for false points that might occur due to reflections or simulation errors. Multiple reflections in Real-world Lidar lead to multipath effects. Simulated Lidar should ideally replicate this effect, but it can also be evaluated if the simulator has mechanisms for handling such errors.
- (e) Material Reflection: Assess how the Lidar interacts with different materials (e.g., metal, glass, asphalt, vegetation). Real-world Lidar behaves differently based on the reflectivity of surfaces, and the simulator should replicate this.
- (f) Weather Effects: High-fidelity simulators should replicate Lidar's performance under adverse weather conditions like rain, fog, or dust. Check for point cloud degradation or noise in such scenarios.
- (g) Sunlight Interference: Bright sunlight can affect real-world Lidar sensors. Verify if the simulator can replicate this effect.
- (h) Clipping Range: Verify if the simulator correctly handles the minimum and maximum detection ranges (i.e., near and far clipping planes).
- (i) Occlusion Handling: Evaluate how well the simulator handles occlusion, such as objects blocking the Lidar beams, resulting in realistic shadows in the point cloud.
- (j) Moving Objects: Check how well the Lidar sensor detects and tracks dynamic objects like cars, pedestrians, or cyclists. Look at the smoothness and accuracy of point cloud generation for moving entities.
- (k) Deformation Effects: Evaluate how Lidar perceives moving or deforming objects (e.g., vegetation bending, tree swaying), which could introduce noise or artifacts in the real world.

- (l) GPU support: Check if the Lidar data is processed with GPU. An efficient simulator should use GPU to accelerate the Lidar data computation.
- (m) Point Cloud Format: Compare the simulator Lidar point cloud format with the real Lidar point cloud format.

Discussion. CARLA supports all the listed sensors, while AWSIM does not support depth cameras and event/DVS cameras.

1. GNSS: AWSIM only output position, no rotation, and covariance.
2. IMU: AWSIM has no noise model.
3. RGB Camera: Table 4 indicates that both simulators cover the essential post-effects for realistic camera simulations, but AWSIM has broader support for advanced effects. CARLA has some issues with anti-aliasing and does not support glare and halo. Both simulators do not support sharpening.
4. Lidar: Table 5 shows that AWSIM provides more configurable parameters, including different noise models, and supports material reflection. Hence, the Lidar intensity varies not just with the distance between the sensor and the obstacle but also with the obstacle material. Moreover, AWSIM Lidar computation has GPU support, allowing it to process denser point clouds at an acceptable frequency; AWSIM can also save the point clouds in XYZIRCAEDT (XYZ position, intensity, return type (including big-endian), channel (or ring), azimuth, elevation, distance) format rather than XYZIR format to accelerate the computing process. This acceleration is achieved by preprocessing the derived fields: A (azimuth), E (elevation), and D (distance). CARLA saves the point clouds in XYZI (XYZ position, intensity) format, which contains less information compared with AWSIM and real Lidar, like Velodyne's XYZIRT (XYZ position, intensity, ring (or channel), time) format.
5. Radar: The Radar result is similar to the Lidar result; AWSIM provides more configurable parameters, including different noise models, and has GPU support for Radar computation. AWSIM can also save the point clouds in a format with more information than CARLA.
6. V2X (Vehicle-to-Everything)/V2I (Vehicle-to-Infrastructure): AWSIM provides a V2I sensor to receive the traffic light signals around the ego vehicle, while CARLA offers a more structured V2X sensor with custom messages. However, this sensor is only available in the latest DEV version.

3.3.3. Environmental Interactions

Description. Simulators must allow controlled, reproducible environmental effects, such as lighting conditions, weather, and road surface properties, ensuring that perception algorithms developed in simulation work in real-world scenarios. Weights are based on the probability and engagement of environmental effects.

Discussion. CARLA offers a dynamic weather API to simulate night, cloudy, rainy, and foggy environments but does not support the snowy effect. AWSIM provides no API for weather, but the Unity editor has customizable third-party weather assets and is easy to use. UE4 has configurable, high-quality, built-in atmospheric and weather effects systems with a steep learning curve. Hence, for custom weather creation, Unity is more accessible for beginners to start with basic weather (via particles or assets). Still, in terms of complex weather creation, Unity requires a lot of scripting, so UE4 is a better choice with its minimal reliance on third-party assets. Unfortunately, all the weather effects, such as fog, snow, and rain in UE4 and Unity, can not affect Lidar. Both CARLA and AWSIM have configurable lighting conditions, pedestrian lights, and traffic lights, but AWSIM's sun object relies on third-party assets; CARLA's pedestrian lights are circle shapes while AWSIM's pedestrian lights are human shapes. The supported road users include pedestrians, cyclists/motorcyclists, cars, and trucks. CARLA has all of them, while AWSIM supports all except cyclists/motorcyclists.

3.4. Scenario Creation and Testing

Compare the ease of creating, configuring, and testing different driving scenarios, including the various scenarios supported (e.g., urban, highway, off-road) and the complexity of scenario setups.

3.4.1. Scenario Construction

Description. Having API to help with scenario generation is vital for speeding up the ADS test process.

Discussion. Evaluate the ease of importing and creating a new map, sensor, and vehicle.

1. New map: Adding a new map has four main processes:
 - (a) Generation: Both use RoadRunner (free for students) or TrueVision (free) exporting in fbx, obj

- (b) Importation: CARLA must build from the source to edit the map. Another importation method, the docker, requires 700 GB of space and cannot be edited after the RoadRunner editing. CARLA takes a lot of time to import since UE4 needs to convert every mesh to an Unreal asset. AWSIM only needs to drag the map into the editor; the map stays in the fbx format.
- (c) Customization: Both can collaborate with multiple people on the same map simultaneously. We may need to fix some RoadRunner assets not supported by UE4 and Unity (e.g., missing materials and meshes). From testing, Unity has better compatibility, so less post-processing is needed.
- (d) Navigation: RoadRunner generates OpenDrive (in xodr format) automatically, and CARLA supports OpenDrive, so it can use the road tags information from OpenDrive to do the random vehicle and pedestrian navigation, no manual configuration is needed. AWSIM only supports Lanelet2 (in OSM format), so manual configuration is required (draw lanes in Lanelet2). However, a team from TUM (Technical University of Munich) published a tool called CommonRoad Scenario Designer [32] that efficiently converts OpenDrive to Lanelet2.

CALRA also supports large maps by cutting them into 2km by 2km tiles, which AWSIM does not have. Both CARLA and AWSIM support OpenScenario for random and route vehicle traffic generation.

2. New vehicle: The main steps to add a new vehicle:

- (a) Prepare Vehicle Model: Obtain or create a 3D vehicle model from 3D modeling software like Blender and ensure the model is divided into vehicle parts based on materials and textures (like wheels, doors, lights, etc.). CARLA takes a few more steps, including importing and binding the skeleton bones, naming the parts and textures in the correct format, and creating separate physical asset mesh and ray cast sensor mesh (with more details).
- (b) Import Model into the Game Engine and Configure Physics: Import from common 3D model formats like fbx and obj, and set up the vehicle's physics by defining parameters like mass, center of mass, and collision boxes. CARLA has additional animation to configure.
- (c) Configure Wheel: set up the front and rear wheel parameters like collision boxes and suspension.

- (d) Configure Vehicle Lights: CARLA has templates for all vehicle lights, such as left blinkers, hazard lights, and reverse lights, but there is no tutorial. AWSIM also supports all vehicle lights and has an easy-to-implement tutorial.
- 3. New Sensors With Templates: CARLA and AWSIM have basic templates for Lidar, cameras, radar, IMU, and GNSS. AWSIM has more Lidar presets, including 3D models and parameters. CARLA has no 3D sensor models, only functions.
- 4. New Sensors Without Templates: For creating sensors not mentioned above, AWSIM has no tutorials, and CARLA has a complicated tutorial.

3.4.2. Scenario Library and Traffic Manager

Description. A rich scenario library simplifies simulation, enabling quicker setup and collaboration. It should include predefined maps, vehicle models, and pedestrian models. We count the number of sample maps, vehicles, and pedestrians to test the richness of the scenario library. For traffic scenarios, simulators should support traffic aids like traffic lights and road signs, which help regulate traffic.

Discussion. As shown in table 3, CARLA has a significantly richer library scenario than ASWIM. Both CARLA and AWSIM support random traffic and route traffic. CARLA uses road information in the OpenDrive file for vehicle and pedestrian motion (random and route motion), including lane type, lane rules, junctions, and traffic lights. AWSIM, on the other hand, needs more manual editing; even with a predefined Lanelet2 file, AWSIM can only extract vehicle lane rules and has to configure pedestrian motion (only straight line back and forth motion), junction, and traffic lights manually. However, AWSIM has more realistic traffic intersection behaviors by providing a yielding priority script (e.g., right-turn vehicles at the intersection should give way to the ones driving straight and turning left). Besides the basic traffic management system, CARLA provides aggressive driver configuration; one can tune the mistake percentage of the vehicle NPCs to simulate real drivers. CARLA also optimizes the traffic in large maps by the NPCs' auto-activating script in tiles; only active NPCs are included in the physics computation.

3.5. User-Friendliness

3.5.1. UI functionality

Description. Every simulator needs to be able to run simulations in a user-defined environment. A robust UI allows for the easy creation of simulations by drag-and-drop controls on models. A good UI can also provide live data more visually, which can help researchers understand and debug simulation runs.

Discussion. Both UE4 and Unity support modular editing. The former is called a blueprint, while the latter is called a prefab. Blueprints and Prefabs are core tools that allow developers to create reusable assets with customized properties, logic, and behaviors. UE4's Blueprint system offers robust, highly flexible options for complex game logic and behavior, which is ideal for detailed systems. Unity's prefab system is generally easier to manage and modify, especially for new users.

CARLA has Rviz and Carlaviz (web visualizer) for data visualization, while AWSIM has Rviz and native display (Unity editor).

3.5.2. Learning Curve

Description. How hard it is to set and learn basic operations.

Discussion. Installation: CARLA is slow to install, especially when built from the source (4 hours or more). However, AWSIM in Unity Editor can be done in less than 1 hour. Regarding editor setup, CARLA has more steps to set up and build the UE4 editor and is more likely to fail when compiling.

Basic Operations: Both CARLA and AWSIM have tutorials for basic operations that are easy to follow and get the demos running. Advanced Operations: For advanced operations, especially adding custom maps and vehicles, Unity's prefab system takes fewer steps to customize than UE4's Blueprint system.

3.5.3. Community and Support

Description. Evaluate the strength of the community and the availability of documentation, tutorials, and support resources, including how active the user community is and the responsiveness of developers to issues and feature requests.

Table 6: CARLA vs. AWSIM Hardware Requirements

Minimum Requirements	CARLA	AWSIM
CPU	4 cores	6 cores and 12 thread
GPU	6 GB GPU	RTX 2080Ti
Storage	130 GB	18 GB
Operating System	Ubuntu 18.04, 20.04; Windows 10, 11	Ubuntu 22.04; Windows 10, 11

Discussion. Documentation Depth: CARLA offers more extensive and well-structured documentation, including comprehensive API references and tutorials from basic to advanced (likely due to its extended presence in the field and broader adoption). In contrast, AWSIM has limited API references and basic tutorials. Community and Supplemental Resources: CARLA has a larger community (more GitHub stars) and a more comprehensive array of supplemental resources (especially research papers). AWSIM is still growing in these areas. Ease of Finding Information: CARLA’s well-organized documentation and active community forums make it easier for developers to find solutions (more GitHub issues). AWSIM’s resources are still developing, so users might need to rely more on direct experimentation. This comparison highlights the maturity of CARLA’s documentation relative to AWSIM, acknowledging that AWSIM’s resources are likely to expand as it continues to develop.

3.5.4. Cost and Licensing

Description. Compare the cost structures (not all researchers can afford expensive and high-end devices), licensing models, and any potential restrictions on using each simulator for commercial or research purposes.

Discussion. CARLA and AWSIM are open-source, but RoadRunner (the tool used to create the map for them) is not. Researchers in most universities have MATLAB campus-wide licenses, including RoadRunner. Buying new assets online for Unity and UE4 also has a cost. As described in table 6, AWSIM requires a higher CPU, GPU, and operating system but less storage. Although CARLA and AWSIM support Windows systems, Windows does not have ROS, so Linux is the primary platform for AV simulator users.

3.6. Extendability

Assess how easy it is to integrate with other tools or frameworks (e.g., ROS).

3.6.1. ROS Integration

Description. ROS is vital to each simulator because it is a widely used robotics middleware. We feel it is most helpful if a simulator has ROS support available on installation or through a plugin (e.g., Abdelhamed et al. manage to design a software-in-the-loop (SIL) framework based on the gazebo_ros_pkgs plugin [33]).

Discussion. CARLA supports ROS2 Foxy; CARLA 0.9.13 is the latest version that supports the official ROS bridge. AWSIM supports ROS2 Humble, which is newer than Foxy and has more functionalities.

3.6.2. Integration with Machine Learning/AI Frameworks

Description. Assess how well each simulator integrates with popular machine learning frameworks (e.g., TensorFlow, PyTorch) and algorithms (e.g., object detection, semantic segmentation) for training and testing autonomous driving models.

Discussion. Sim detectors are the detectors that only exist in the simulator. Collision and lane invasion detectors evaluate AV systems, and obstacle detectors simulate real-world proximity sensors (e.g., ultrasonic, electromagnetic). CARLA supports all the sim detectors, while AWSIM has not provided any script for AV system evaluation. Camera annotation sensors provide ground-truth object labels in images. These labels are essential for training perception AI algorithms. CARLA generates these labels using predefined object tags in the simulator. AWSIM does not have these sensors, but Unity has a perception module supporting all these functions with detailed tutorials. Like the camera annotation sensors, Lidar annotation sensors provide ground-truth object labels in point clouds. These labels are used to train Lidar-based AI algorithms. Both CARLA and AWSIM support semantic segmentation Lidar annotation.

3.6.3. Programming Languages

Description. The more programming languages supported, the more accessible a simulator may be to a broad user base.

Discussion. CARLA has Python and C++ support for standalone UE4 executables. AWSIM does not support scripting for standalone Unity executables but has a configurable GUI (e.g., function buttons in the game window). UE4 editor has built-in C++ support. Unity editor has built-in C# support.

4. Conclusion

This paper conducted an in-depth, comprehensive comparative analysis on the same map in UE4-based CARLA and Unity-based AWSIM to compare their simulator performance for autonomous driving. Hence, it compared their game engines behind. CARLA won the overall performance. However, AWSIM has advantages in several aspects, including faster iteration cycles (benefit from Unity) and more accurate Lidar simulation. So it is more suitable for Lidar-based neural networks, Autoware and Baidu Apollo testing. At the same time, CARLA is still the most suitable simulator for systematic end-to-end testing.

This paper systematically categorized comparison criteria (both qualitative and quantitative) for open-source autonomous driving simulators. Future researchers can compare other simulators with the listed criteria in this paper and adapt the weight according to their needs.

Future works involve testing end-to-end algorithms, including camera-based and Lidar-based neural networks, and modularized algorithms, including Autoware and Baidu Apollo.

References

- [1] W. Shi, L. Liu, Autonomous driving simulators, in: W. Shi, L. Liu (Eds.), *Computing Systems for Autonomous Driving*, Springer International Publishing, 2021, pp. 143–156. doi:10.1007/978-3-030-81564-6_6.
URL https://doi.org/10.1007/978-3-030-81564-6_6
- [2] J. Seymour, D.-T.-C. Ho, Q.-H. Luu, An empirical testing of autonomous vehicle simulator system for urban driving, in: *2021 IEEE International Conference on Artificial Intelligence Testing (AITest)*, 2021, pp. 111–117. doi:10.1109/AITest52744.2021.00031.
URL <https://ieeexplore.ieee.org/document/9564102>
- [3] Y. Li, W. Yuan, S. Zhang, W. Yan, Q. Shen, C. Wang, M. Yang, Choose your simulator wisely: A review on open-source simulators for autonomous driving (2024) 1–19Conference Name: *IEEE Transactions on Intelligent Vehicles*. doi:10.1109/TIV.2024.3374044.
URL <https://ieeexplore.ieee.org/document/10461065>

- [4] Simulation city: Introducing waymo's most advanced simulation system yet for autonomous driving.
URL <https://waymo.com/blog/2021/07/simulation-city>
- [5] Cognata | AV & ADAS real data & simulation tools for AI.
URL <https://www.cognata.com/>
- [6] AVSimulation | AVS : Automotive simulation software and solutions (2024).
URL <https://www.avsimulation.com/en/>
- [7] Simcenter prescan software simulation platform.
URL <https://plm.sw.siemens.com/en-US/simcenter/autonomous-vehicle-solutions/prescan/>
- [8] Mechanical simulation corporation.
URL <https://www.carsim.com/>
- [9] CarMaker | IPG automotive.
URL <https://www.ipg-automotive.com/en/products-solutions/software/carmaker/>
- [10] RoadRunner scenario.
URL <https://au.mathworks.com/products/roadrunner-scenario.html>
- [11] NVIDIA DRIVE sim.
URL <https://developer.nvidia.com/drive/simulation>
- [12] N. Koenig, A. Howard, Design and use paradigms for gazebo, an open-source multi-robot simulator, in: 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566), Vol. 3, 2004, pp. 2149–2154 vol.3. doi:10.1109/IROS.2004.1389727.
URL <https://ieeexplore.ieee.org/document/1389727>
- [13] O. Michel, Cyberbotics ltd. webots™: Professional mobile robot simulation 1 (1) (2004) 5, publisher: SAGE Publications. doi:10.5772/5618.
URL <https://doi.org/10.5772/5618>

- [14] S. Shah, D. Dey, C. Lovett, A. Kapoor, AirSim: High-fidelity visual and physical simulation for autonomous vehicles (2017). [arXiv:1705.05065\[cs\]](#), doi:10.48550/arXiv.1705.05065.
URL <http://arxiv.org/abs/1705.05065>
- [15] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, V. Koltun, CARLA: An open urban driving simulator (2017). [arXiv:1711.03938\[cs\]](#), doi:10.48550/arXiv.1711.03938.
URL <http://arxiv.org/abs/1711.03938>
- [16] G. Rong, B. H. Shin, H. Tabatabaee, Q. Lu, S. Lemke, M. Možeiko, E. Boise, G. Uhm, M. Gerow, S. Mehta, E. Agafonov, T. H. Kim, E. Sterner, K. Ushiroda, M. Reyes, D. Zelenkovsky, S. Kim, LGSVL simulator: A high fidelity simulator for autonomous driving, in: 2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC), 2020, pp. 1–6. doi:10.1109/ITSC45102.2020.9294422.
URL <https://ieeexplore.ieee.org/abstract/document/9294422>
- [17] tier4/AWSIM, original-date: 2022-10-14T00:48:24Z (2024).
URL <https://github.com/tier4/AWSIM>
- [18] X. Hu, S. Li, T. Huang, B. Tang, R. Huai, L. Chen, How simulation helps autonomous driving: A survey of sim2real, digital twins, and parallel intelligence 9 (1) (2024) 593–612, conference Name: IEEE Transactions on Intelligent Vehicles. doi:10.1109/TIV.2023.3312777.
URL <https://ieeexplore.ieee.org/document/10242366?denied=>
- [19] A. Kadian, J. Truong, A. Gokaslan, A. Clegg, E. Wijmans, S. Lee, M. Savva, S. Chernova, D. Batra, Sim2real predictivity: Does evaluation in simulation predict real-world performance? 5 (4) (2020) 6670–6677. [arXiv:1912.06321\[cs\]](#), doi:10.1109/LRA.2020.3013848.
URL <http://arxiv.org/abs/1912.06321>
- [20] M. C. Figueiredo, R. J. F. Rossetti, R. A. M. Braga, L. P. Reis, An approach to simulate autonomous vehicles in urban traffic scenarios, in: 2009 12th International IEEE Conference on Intelligent Transportation Systems, 2009, pp. 1–6, ISSN: 2153-0017. doi:10.1109/ITSC.2009.5309524.
URL <https://ieeexplore.ieee.org/document/5309524>

- [21] M. Quigley, B. Gerkey, K. Conley, J. Faust, T. Foote, J. Leibs, E. Berger, R. Wheeler, A. Ng, ROS: an open-source robot operating system.
- [22] N. Kalra, S. M. Paddock, Driving to safety: How many miles of driving would it take to demonstrate autonomous vehicle reliability?, in: Driving to Safety, How Many Miles of Driving Would It Take to Demonstrate Autonomous Vehicle Reliability?, RAND Corporation, 2016, pp. 1–16. URL <https://www.jstor.org/stable/10.7249/j.ctt1btc0xw.1>
- [23] J. Fadaie, The state of modeling, simulation, and data utilization within industry: An autonomous vehicles perspective (2019). arXiv:1910.06075[cs], doi:10.48550/arXiv.1910.06075. URL <http://arxiv.org/abs/1910.06075>
- [24] ApolloAuto/apollo, original-date: 2017-07-04T19:03:31Z (2025). URL <https://github.com/ApolloAuto/apollo>
- [25] autowarefoundation/autoware, original-date: 2015-08-24T23:17:57Z (2025). URL <https://github.com/autowarefoundation/autoware>
- [26] M. Violante, A COMPLETE END-TO-END SIMULATION FLOW FOR AUTONOMOUS DRIVING FRAMEWORKS.
- [27] P. Kaur, S. Taghavi, Z. Tian, W. Shi, A survey on simulators for testing self-driving cars, in: 2021 Fourth International Conference on Connected and Autonomous Driving (MetroCAD), 2021, pp. 62–70. doi:10.1109/MetroCAD51599.2021.00018. URL <https://ieeexplore.ieee.org/document/9499331>
- [28] J. Zhou, Y. Zhang, S. Guo, Y. Guo, A survey on autonomous driving system simulators, in: 2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), 2022, pp. 301–306. doi:10.1109/ISSREW55968.2022.00084. URL <https://ieeexplore.ieee.org/document/9985152>
- [29] G. Scapicchi, AR-sim: A high-fidelity 3d simulator for autonomous race-cars (2024). URL <https://webthesis.biblio.polito.it/31840/>

- [30] A. Stocco, B. Pulfer, P. Tonella, Mind the gap! a study on the transferability of virtual versus physical-world testing of autonomous driving systems 49 (4) (2023) 1928–1940, conference Name: IEEE Transactions on Software Engineering. doi:10.1109/TSE.2022.3202311. URL <https://ieeexplore.ieee.org/abstract/document/9869302>
- [31] H. Haghighi, M. Dianati, V. Donzella, K. Debattista, A unified generative framework for realistic lidar simulation in autonomous driving systems (2024). arXiv:2312.15817[cs], doi:10.48550/arXiv.2312.15817. URL <http://arxiv.org/abs/2312.15817>
- [32] M. Althoff, S. Urban, M. Koschi, Automatic conversion of road networks from OpenDRIVE to lanelets, 2018. URL <https://mediatum.ub.tum.de/1449005>
- [33] A. AbdelHamed, G. Tewolde, J. Kwon, Simulation framework for development and testing of autonomous vehicles, in: 2020 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS), 2020, pp. 1–6. doi:10.1109/IEMTRONICS51293.2020.9216334. URL <https://ieeexplore.ieee.org/abstract/document/9216334>

5. Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used Grammarly Pro AI and ChatGPT in order to improve writing. After using this tool, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

